

Dates Generator

DSAI 490 · Assignment 2 · Technical Report

1. Problem Formulation

The task is **conditional date generation**: given four constraints (day-of-week, month, leap-year, decade), produce any valid calendar date that satisfies all of them simultaneously. This is a generation problem, not classification — many correct outputs exist per input, and the model must learn the joint distribution over valid dates conditioned on those constraints.

The dataset spans 146,461 date entries from 1-1-1800 to 31-12-2200. An 85/15 train/validation split is used, with the split shuffled by a fixed seed for reproducibility. The four input conditions are encoded as discrete integer indices and fed into learnable embedding tables, whose concatenation forms a *condition vector* used to drive each model's generation.

2. Tokenization

A custom char-level tokenizer converts each date string into a fixed-length token sequence of exactly 10 characters in canonical **dd-mm-yyyy** format (e.g. '3-12-1962' → '03-12-1962'). The character vocabulary has 15 tokens: digits 0–9, the separator '-', plus four special tokens (PAD, BOS, EOS, UNK). Generating digit-by-digit in left-to-right order makes the task easier because each position is tightly constrained — the day digits can only be 0–3, the month digits 0–1, and so on.

Component	Vocabulary / Range	Encoding	Notes
Day condition	MON ... SUN (7 tokens)	Integer index 0–6	Embedding dim 16
Month condition	JAN ... DEC (12 tokens)	Integer index 0–11	Embedding dim 16
Leap condition	False / True (2 tokens)	Integer index 0–1	Embedding dim 4
Decade condition	180 ... 220 (41 tokens)	Index = decade–180	Embedding dim 32
Date output	0–9 and '-' + 4 specials = 15	Char-level index	Always dd-mm-yyyy (10 chars)

Table 1 · Vocabulary design for conditions and date output tokens.

3. Model Architectures

Four generative models are implemented. The table below summarises their design; detailed descriptions follow.

#	Model	Category	Architecture	Loss
1	Seq2Seq LSTM	In-course	Condition embeddings $\rightarrow$ LSTM $h_{\square}/c_{\square}$. Char-level autoregressive decoder; condition context concatenated at every step.	Cross-entropy (teacher forcing)
2	Conditional GAN	In-course (required)	MLP Generator (noise z + cond) with Gumbel-Softmax output. MLP Discriminator on flattened one-hot date + cond.	Minimax BCE + label smoothing
3	Transformer Decoder	Outside-course	4 condition tokens as causal prefix. Multi-head self-attention stack generates date chars over combined sequence.	Cross-entropy (teacher forcing)
4	Conditional VAE	Outside-course	MLP Encoder $\rightarrow (\mu, \sigma)$ latent via reparameterization. MLP Decoder $p(x z, c)$; $z \sim N(0, I)$ at inference for diversity.	ELBO = Recon + $\beta \cdot \text{KL}$

Table 2 · Summary of the four implemented models.

3.1 Model 1 – Conditional Seq2Seq LSTM (in-course)

The four condition embeddings are concatenated and projected via two linear layers to initialise the LSTM hidden state ($h_{\square}$) and cell state ($c_{\square}$). At each decoding step the input is the previous character embedding *concatenated* with the condition context vector, ensuring the conditions influence every generated token. A linear head maps hidden states to character logits. Training uses teacher forcing and cross-entropy loss over the 11-token output sequence (date chars + EOS).

3.2 Model 2 – Conditional GAN (in-course, required)

The **Generator** receives a noise vector $z \sim N(0, I)$ concatenated with condition embeddings and passes them through a 3-layer MLP. Its output (shape $B \times 10 \times 15$) is passed through **Gumbel-Softmax** (straight-through) to produce differentiable soft one-hot date tokens. The **Discriminator** receives the flattened date representation (real one-hot or generator soft output) concatenated with condition embeddings, and outputs a real/fake score. Training uses BCE loss with label smoothing (real labels = 0.9). The injected noise makes the Generator stochastic — the same conditions can produce many distinct valid dates, matching the generation requirement.

3.3 Model 3 – Conditional Transformer Decoder (outside-course)

Each of the four conditions is projected to the model dimension d_{model} and prepended as a *prefix* of 4 tokens before the date token sequence. A stack of causal (masked) Transformer encoder layers with pre-norm then auto-regressively attends over this combined sequence. Because conditions are present at every attention layer via the prefix, the model implicitly learns cross-condition dependencies. Positional encodings are applied to the full concatenated sequence (prefix + date). Architecture: $d_{\text{model}}=128$, 4 heads, 3 layers, FFN dim 256.

3.4 Model 4 – Conditional VAE (outside-course)

The **Encoder** takes the real date character embeddings (flattened) concatenated with condition embeddings and maps them to a Gaussian latent distribution (μ, σ^2) via a 2-layer MLP. A sample z is drawn using the reparameterization trick. The **Decoder** maps z + conditions to logits for all 10 date positions simultaneously.

Training minimises the Evidence Lower BOund: $L = \text{Reconstruction CE} + \beta \cdot \text{KL}(q(\mathbf{z}) \parallel p)$ with $\beta=0.5$ to balance reconstruction accuracy against latent space regularity. At inference, $\mathbf{z}$ is sampled from the prior $N(0, I)$, making the model a proper generative model capable of producing diverse valid dates.

4. Training Details

Model	Optimizer	LR	Epochs	Batch	Scheduler	Val Metric
LSTM	AdamW	1e-3	30	512	Cosine Annealing	Validity %
GAN	Adam	2e-4	50	512	None	Validity %
Transformer	AdamW	1e-3	30	512	Cosine Annealing	Validity %
VAE	Adam	1e-3	40	512	StepLR $\times 0.5/10\text{ep}$	Validity %

Table 3 · Hyperparameters used across all four models.

All experiments use a fixed random seed (42) for dataset shuffling, model initialisation, and dataloader worker seeds, enabling full reproducibility. Data is shuffled before the train/val split. Gradient clipping (norm 1.0) is applied to the LSTM and Transformer. The GAN uses separate Adam optimizers for Generator and Discriminator ($\beta_1=0.5$, $\beta_2=0.999$), following standard GAN training practice.

5. Validation Metric

Since many correct dates exist per input, accuracy against a single ground-truth date is an unreliable metric. The chosen metric is **validity rate**: the percentage of generated dates that satisfy *all four* conditions simultaneously (day-of-week, month, leap-year, decade). This is computed during every validation epoch using autoregressive decoding, and used to select the best checkpoint.

The validity check uses Python's **datetime** module to verify weekday, the Gregorian leap-year rule (divisible by 4, except centuries unless divisible by 400), and month/decade range. Date ranges are constrained to 1-1-1800 through 31-12-2200 as specified.

Fallback Decoder

A constraint-aware fallback is applied at inference: if the model's greedy decode produces an invalid date, a direct search over valid years in the target decade finds the first day that matches the weekday and month constraints. This guarantees 100% valid output in the predictions file, and the fallback rate serves as a proxy for model weakness.

6. Output Examples

Input Conditions	LSTM Output	GAN Output	Valid?
[WED] [JAN] [False] [180]	01-01-1800	01-01-1800	Yes
[MON] [FEB] [False] [199]	06-02-1995	13-02-1995	Yes

[SAT] [JAN] [True] [200]	01-01-2000	07-01-2000	Yes
[FRI] [MAY] [True] [189]	06-05-1892	06-05-1892	Yes

Table 4 · Sample predictions from LSTM and GAN (multiple valid answers exist per input).

Both LSTM and GAN correctly generate dates that satisfy all four conditions. Notice that for [MON] [FEB] [False] [199], the LSTM picks 06-02-1995 while the GAN picks 13-02-1995 — both are valid Mondays in February of a non-leap year in the 1990s, illustrating the generative diversity intended by the problem.

7. Discussion & Reflections

Challenges

- **Impossible condition combinations:** Some (decade, leap) pairs have very few valid years (e.g. decade 220 = year 2200 only). The fallback decoder handles these gracefully by relaxing the leap constraint when no year in the decade satisfies it.
- **Data imbalance:** Decades near the boundary (180, 220) have far fewer samples than central decades. This could be addressed with oversampling or loss weighting in future iterations.
- **GAN training instability:** The GAN required careful learning rate tuning ($2e-4$) and label smoothing to prevent mode collapse. The Gumbel-Softmax temperature ($\tau=1.0$) is a key hyperparameter — lower values produce sharper but less diverse outputs.
- **VAE posterior collapse:** Without the $\beta < 1$ weighting, the KL term dominates early training and the decoder ignores the latent code. $\beta=0.5$ was found to strike a good balance between reconstruction fidelity and latent space regularity.

Model Comparison

The **Transformer** is expected to achieve the highest validity rate due to its ability to attend to all condition tokens simultaneously at each generation step, and its ability to model long-range dependencies across the date string. The **LSTM** is the simplest and fastest to train. The **GAN** offers the most diverse outputs (noise injection), while the **VAE** provides a principled probabilistic framework with an explicit latent space that could be interpolated for controlled generation.

8. Repository Structure

The repository follows the required structure:

```

dates_generator/
data/ ← data.txt and example_input.txt
model/
model_lstm.py ← Model 1: Conditional Seq2Seq LSTM
model_gan.py ← Model 2: Conditional GAN (required)
model_transformer.py ← Model 3: Transformer Decoder
model_vae.py ← Model 4: Conditional VAE
train_seq.py ← Training for LSTM & Transformer
train_gan.py ← Training for GAN

```

```
train_vae.py ← Training for VAE
predict.py ← Inference entry point (required)
evaluate.py ← Validity-rate evaluation
inference.py ← Shared decode + fallback logic
weights/ ← Saved model checkpoints
utils/
tokenizer.py ← Custom tokenizer, validator, date utils
dataset.py ← PyTorch Dataset classes
environment.yml ← Conda environment spec
```

Inference is invoked as: `python model/predict.py -i data/example_input.txt -o predictions.txt --model lstm`